Tokens Are Progress: Serving LLM Coding Agents on ARC-AGI-2 and ARC-AGI-3

Team 復活の混テキスト*

ARC Prize 2026 Paper Track · Track: ARC-AGI-2 · documented submission 57852801

Abstract

We document two ARC Prize 2026 entries built on one design: a general-purpose mixture-of-experts LLM reasons inside a persistent Python sandbox, and most of the engineering goes into the inference server. Our ARC-AGI-2 submission (grid tasks; four L4 GPUs; 12 hours) scored 73.47 on the public leaderboard (rank 25 of 3,543 teams). Our ARC-AGI-3 submission (interactive games; one RTX PRO 6000; 9 hours) scored 46.20 (team best 46.82, rank 15 of 4,923). Both follow from a budget model: under a fixed wall clock, score is roughly the tokens delivered per task times the score each token buys. Decoding is memory-bound, so a few levers set the token rate. We report each lever's measured effect, the faster changes that lost because each token bought less, and why different levers paid on a bandwidth-bound and a capacity-bound GPU.

1 Introduction

ARC-AGI-2 [1] asks for grid transformations inferred from a few demonstrations. ARC-AGI-3 [2] presents unknown interactive games: an agent sees a 64×64 frame, acts, and is scored on levels cleared relative to human action counts. Both test adaptation to novelty [3] offline, on fixed hardware, against a wall clock. Recent award-winning work changed the learner [4, 5]; we ask how far a general LLM agent goes when its serving is engineered for the budget. Our documented submission is the ARC-AGI-2 entry. The ARC-AGI-3 entry, made by a larger team that includes all five authors, is a second case for the same budget model on a different GPU.

Adopting a code-writing agent raised what each token bought on both benchmarks. After that, the largest leaderboard steps came mostly from delivering more tokens per hour. We contribute a budget model that predicts which serving changes pay (§3), two serving stacks for one 512-expert MoE with measured effects (§5), negative results (§7), and an evaluation protocol for leaderboards whose repeated scorings of identical code differ by up to 10 points (§6).

2 Related work

ARC solvers. The ARC Prize 2024 entries combined program search with test-time training [6, 7], and the winners scored each candidate answer under several augmented views of the task [8]. In 2025, award-winning papers trained a small recursive network from scratch [4] and improved an LLM program synthesizer on its own search traces [5]; the top Kaggle entry used an ARC-specialized 4B Qwen3 model [9], and the technical report surveys the field [10]. We train no solver. Our ARC-AGI-2 entry began from the NVARC model (Table 2) and moved to a general LLM that writes and tests programs.

Agents on ARC-AGI-3. Our ARC-AGI-3 agent builds on two public harnesses: Tufa Labs' Duck harness, the winning solution of Milestone 1 [11], and D. Franzen's Milestone-2 harness, which builds on it [12]. In both, one LLM acts on the game through a single Python tool. §4 lists what we added.

Efficient MoE serving. Each lever we use comes from prior work. PagedAttention [13] makes KV memory the unit of concurrency. Speculative decoding [14] and multi-token-prediction heads [15] raise the tokens accepted per step. KV quantization [16] stores more tokens per byte. Quantized GEMM kernels [17, 18] cut weight traffic. REAP [19] prunes experts in one shot, and OEA [20] reduces the distinct experts per batch. We propose no new technique. We contribute measurements of these levers under fixed hardware and a fixed clock, and a model that predicts which of them pays.

*Kaggle members: isakatsuyoshi, konbu17, pompom555, refv00, tanakaai24.

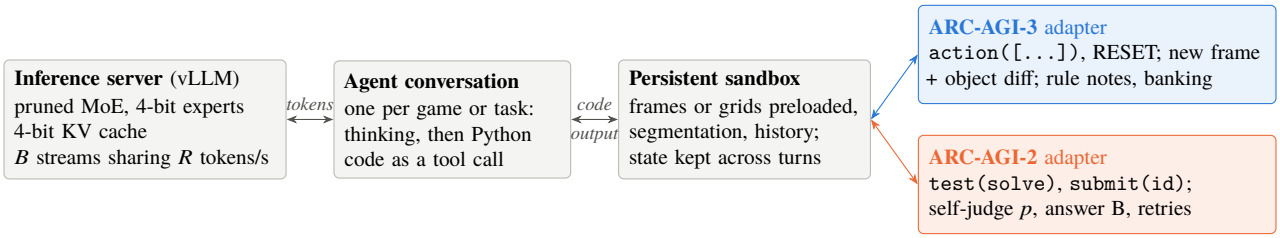


Figure 1: Shared architecture. One server streams tokens to many conversations; each acts only through a persistent Python sandbox. Only the adapters on the right are competition-specific.

3 A budget model

Let n tasks share an aggregate decode rate R (tokens/s) for a time limit T. Each task receives about $N = RT/n$ generated tokens, and

$$\mathbb{E}[\text{score}] \approx \frac{RT}{n} \cdot w, \tag{1}$$

where the *token worth* w is the expected score per token: levels per token on ARC-AGI-3, correct answers per token on ARC-AGI-2. The server sets R; the model, prompt and harness set w. We kept a change only if the product rose.

Eq. (1) is a first-order approximation, since per-task returns are concave. In our 110-game ARC-AGI-3 run, 41 of the 42 games not won had cleared a level, but their last clear came at a median of 40% of their model turns (Figure 3, left). Extra tokens pay most in games that are moving, and harness changes that turn idle turns into progress raise w. On ARC-AGI-2, one pass over 240 tasks costs about 12M tokens, half the budget, and the rest buys retries.

At our batch sizes, MoE decoding is memory-bound. A step reads the experts the batch selects, the dense weights and every active sequence's KV cache:

$$t_{\text{step}} \approx \frac{\sum_\ell D_\ell(B)\, s_e + s_d + \sum_{b=1}^{B} c_b\, s_{kv}}{\text{BW}} + t_{\text{samp}}, \qquad R \approx \frac{a\,B}{t_{\text{step}}}, \qquad B\,\bar{c}\,s_{kv} \le M_{kv}, \tag{2}$$

with $D_\ell(B)$ the distinct experts that B sequences select in layer ℓ, s_e and s_d the bytes of one expert and of the dense weights, c_b a sequence's context, s_{kv} KV bytes per token, a tokens accepted per step, t_{samp} the sampler time and M_{kv} the KV memory. Eq. (2) names five levers: smaller weights, smaller KV (more streams), fewer distinct experts per batch, more accepted tokens per step, and less vocabulary-wide work.

What pruning saves depends on the tokens per step. When a step runs B tokens under uniform top-k routing over E experts, it reads $\mathbb{E}[D] = E\,(1 - (1 - k/E)^B)$ distinct experts per layer. Pruning from 512 to 320 experts frees 37.5% of expert memory for KV. The cut in expert traffic grows with B (here k=10). At 32 tokens per step it is 15% (240 → 204 experts). At 128 tokens, which is 32 streams each verifying 3 draft tokens, it is 33% (471 → 315). On ARC-AGI-2, about 75 sequences decode at once, and pruning from 512 to 256 cuts the traffic by 38% (395 → 243). Opportunistic Expert Activation (OEA) [20] keeps each token's top k_0 experts and fills the rest from experts the batch already uses. At k_0=5 and 320 experts, it reads 127 experts instead of 204 at 32 tokens per step, and 277 instead of 315 at 128, so OEA attacks the traffic term. Real routing is skewed; only the ratios matter.

The binding constraint decides which lever pays. On the 96-GB RTX PRO 6000 with 4-bit KV, 32 streams used at most 67% of the 4.76M-token KV pool (55% on average). Per-step cost was therefore the margin, and OEA, speculative decoding and a cheaper sampler paid. On 24-GB L4s, capacity binds: speculative decoding lost because its draft state took 25% of KV, and the gains came from fitting more conversations.

4 Two agents

ARC-AGI-2: test, judge, retry. Perception, not reasoning, limited accuracy: in 8 failed traces read in full, the model had written the correct rule every time, and in 5 it discarded the rule after misreading a cell. Each task is therefore one conversation with a persistent interpreter holding the grids, following the one-tool pattern of the public duck-harness [11]. In an earlier harness, this raised single-attempt accuracy from 35.4% to 57.7%. `test(solve)` stores a candidate only

Lever (Eq. 2)	ARC-AGI-3: 1× RTX PRO 6000, 9 h	ARC-AGI-2: 4× L4, 12 h
Engine	vLLM + NVFP4 experts instead of SGLang W4A16: 713 vs ~580 tok/s; LB 27.00 → 30.51	vLLM 0.30, Humming INT4 MoE kernels [18]; pre-converted experts: server ready 905 → 450 s
Expert pruning (REAP [19])	Public REAP-448 checkpoint pruned to 384 at load, 20 streams: 884 vs 713 tok/s at equal levels (70 vs 68); LB 30.51 → 38.15. Later, 512 → 384 calibrated on contexts from all 25 public games (LB 38.14, 39.53), served as 320 from load	512 → 256, calibrated on 1.05M tokens of the agent's own conversations: held-out tasks 65.5% vs 64.8% unpruned; 1.9× tok/s
Weight precision	NVFP4 experts; draft-layer experts FP8 → NVFP4: −1.0 GiB, +3.2% tok/s	INT4 experts (NVFP4 lost accuracy); 8-bit dense layers: +4–8% tok/s, +10% KV
4-bit KV cache	NVFP4: pool 2.9M → 4.76M tokens, 24 → 32 streams; ΔNLL −0.003 (95% CI −0.050, +0.045)	INT4 codes + fitted 8-bit scale per 16 values: ~1.6× KV tokens; ΔNLL +0.0016 vs FP8
Distinct experts per batch	OEA k_0=5 + Marlin MoE kernels [17]: +8–10% tok/s; with a fused routing kernel, about +14% (measured across sessions)	—
Accepted tokens per step	retrained MTP draft head [14, 15], 3 draft tokens: +3% tok/s in replay	not used: draft state took 25% of KV
Sequences in memory	—	TP2×PP2 instead of TP4: 1.8–1.9× sequences; ~75 decode while ~75 wait in 64 GiB of host memory
Recomputation	—	DeltaNet state kept at prompt end and newest generation boundary: recomputed/generated 5.1 → 1.1; +15% finished conversations
Vocabulary-wide work	sampling over 49,152 of 248,320 ids (exact logits on kept ids): sampler 1.57 → 0.54 ms, step −2.6%	—

Table 1: Serving changes, grouped by the term of Eq. (2) they act on. Speeds are local measurements; LB changes are single public draws.

if it reproduces every training output, and `submit(id)` ends the search with answer A. The same conversation is then asked (no thinking, temperature 0) for the probability p that A is wrong, read as the expectation over the digit tokens' probabilities. If $p \geq 0.3$, the model seeks a second rule that fits every training pair but changes a test output (answer B). Freed slots re-solve the riskiest tasks in fresh conversations. Identical grids are merged across conversations and ranked by $vrisk = p \cdot 0.5^{m-1}$, where m conversations produced the grid. Empty second slots are filled by *T1n*: logged sandbox code is re-executed offline to recover intermediate versions of `solve`, ranked by the training pairs they match. In the last 15 minutes, running conversations are harvested the same way.

ARC-AGI-3: playing unknown games. We started from D. Franzen's public Milestone-2 harness [12], which builds on Tufa Labs' Duck harness [11]. Our byte-identical run of it scored 26.11, against 7.55 for our own earlier agent, which we had also developed from the Duck harness. Each turn, the model sees the newest frame (letter-coded text and an image) and has one tool, a Python interpreter with the frame, its connected-component segmentation and the full transition history preloaded. It writes code to diff frames, keep a world model and search (BFS, beam search) over inferred dynamics, then calls `action([...])`, often with a planned sequence. Context is trimmed from about 118k to 58k tokens, keeping reasoning history. A scheduler admits all 110 games and decides which 32 hold a stream.

Our additions: rule notes that survive trimming for the last 8 cleared levels; for click-only games, a prompt asking for a function that predicts each action's effect, used for planning only once it explains every recorded transition; a RESET after 30 analysis steps without a board change; and keepalive RESETs during the 10–14-minute model load. We also turned on the harness's object-level frame diff in each message. Finally, *post-WIN banking*: the scorecard keeps a game's best play, so after a WIN the harness sends RESET to the same environment (it never calls `make()` a second time). It then replays, without model calls, the sequences that cleared each level, stopping at the first board mismatch. A third play replays them with board loops cut.

5 Serving: one model, two GPU budgets

Both stacks serve Qwen3.8-Flash-Next [21] (48 layers: 36 Gated DeltaNet [22], 12 attention; 512 experts, top-10), released under the Qwen Community License 1.0, with vLLM [13]. For ARC-AGI-3 we used lordhansolo's public vLLM build and NVIDIA's NVFP4 checkpoint; for ARC-AGI-2, dineshkumar0705's public FP8 copy (Table 1). Every patch verifies itself at load (SHA-256 pins, logged markers) and has a fail-safe. End to end, the ARC-AGI-3 notebook generated about 1,320 tokens/s over a 9-hour local run. The public baseline's stack measured about 580 in a 40-minute test. The conditions differ, so this comparison is indicative only. On ARC-AGI-2, memory savers, host-memory parking and dropping MTP

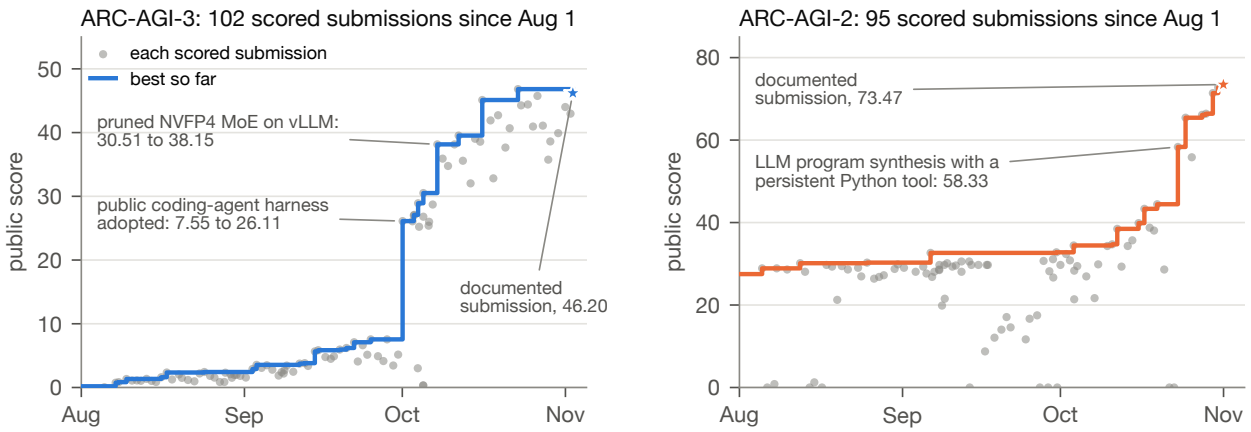


Figure 2: Public score of every scored team submission since August 1 (gray) and the best so far, up to each documented submission (star).

raised a 2-hour 4×L4 test from 363 to 560 tokens/s and from 23 to 47 finished tasks.

6 Evaluation

Generated games for local evaluation. Only 25 ARC-AGI-3 games are public, while the hidden set has 110 games, 55 of which the public leaderboard scores. We therefore generated more games. LLMs wrote games in the official ARCEngine format from a hand-written catalog of 30 generic puzzle mechanics, without seeing any public game's code. A game was kept only if its reference solution replayed to WIN through the official `arc_agi` toolkit, and was used only if a model review, reading its code and probing it in play, marked it usable. The frozen evaluation set lists 25 public games, 33 community-made held-out games and 92 generated games. Its first 110 games match the hidden set in number, which matters when comparing schedulers or speed, where the number of games per stream affects the result. Two caveats apply. Neither the generated nor the community games have human action counts as baselines; generated games use the length of their reference solution, so their scores read lower than on official games. We also did not measure whether scores on these games track scores on the official games.

Local replicas. For ARC-AGI-3 we ran the first 110 games of this set (25 public, 33 community-made, 52 generated) for 9 hours with the submitted notebook, changing only the game source: 48.25 unbanked (first play), 59.27 banked (best of up to three plays), 68 games won. How much of the banking gain the competition credits is unknown. For ARC-AGI-2 we ran arms side by side at equal KV on the 120 public evaluation tasks, often with an orientation-changed copy of each task. A 12-hour real-L4 run of a submitted predecessor scored 9 points above its leaderboard score, so we compared changes, not absolute values.

Repeated scorings of identical code spread by up to 10.1 points. The final ARC-AGI-3 notebook scored 46.20 before the deadline, and 45.32, 43.18 and 53.27 after it (Figure 3, right). We therefore decided with paired per-task differences, progress at equal elapsed time, and speed at matched numbers of running sequences. Figure 2 and Table 2 show the leaderboard path. The paper documents the submissions scored before the deadline; later scorings of the same code appear only as marked extra draws. On ARC-AGI-3, a serving-only change (384 experts, 20 instead of 14 streams) moved scores from 30.51 to 38.15 and 35.91, two scorings of the same code.

7 What did not work: faster, but worth less

The most instructive failures raised R but lowered w (Table 3). Self-distillation is the clearest case. Pruning to 224 experts with LoRA trained toward the 512-expert model recovered untimed accuracy (72.9% vs 68.7% for 256 experts, positive in 5 of 5 pairs), but longer outputs cut L4 concurrency from 37–40 to 32–35 requests. Compression must preserve accuracy per token, not accuracy.

ARC-AGI-3 notebook	public score
public harness, our run	26.11
+ vLLM, REAP-448 NVFP4 checkpoint	30.51
+ 384 experts, 20 streams	38.15, 35.91
+ expert bank calibrated on 25 public games	38.14, 39.53
+ OEA, fused routing, Marlin, 22 streams, hypothesis check	38.57, 45.11, 39.37*
+ 320 experts, 24 streams, stuck reset, banking	46.82, 38.61
final: + 4-bit KV, 32 streams, draft head, compact sampling	**46.20**, 45.32*, 43.18*, 53.27*

ARC-AGI-2 notebook	public score
NVARC recipe [9] + our selector	30.28
Flash-Next (256 experts, INT4) writing programs	39.86, 44.44
persistent Python tool, state pinning	58.33
+ example pictures, CPU KV offload	65.42
+ self-judge, answer B, retries	66.11
+ 4-bit KV, conversation cache, 60 streams	71.39, 72.78
final: fitted INT4 KV, p expectation, vrisk, T1n, harvesting, 150 streams	**73.47**, 74.86*, 73.61*, 71.25*

Table 2: Leaderboard path. Rows with several scores list every scoring of that row's code, in order; * scored after the deadline. Bold: the documented submissions.

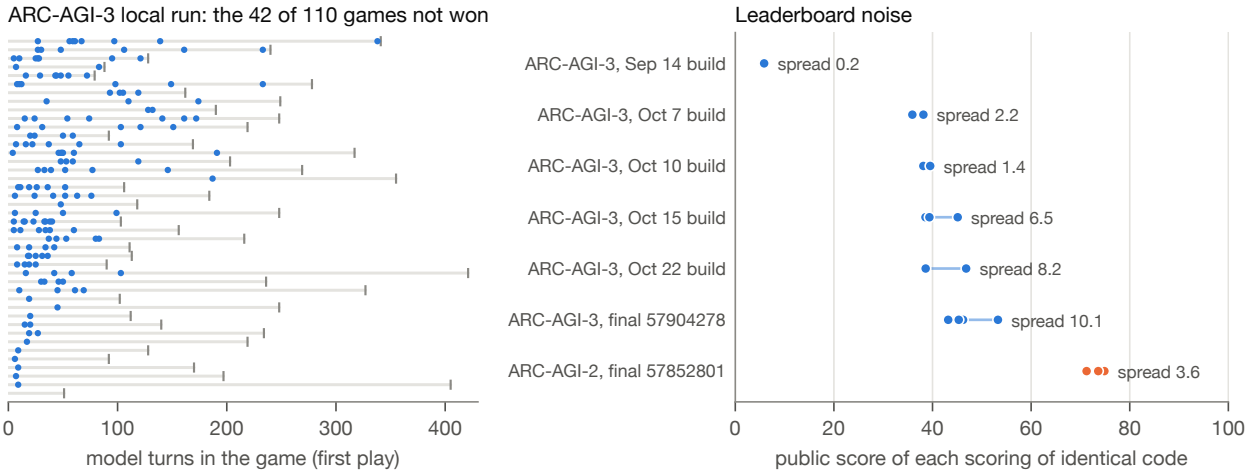


Figure 3: *Left*: level clears (blue) of the 42 ARC-AGI-3 games not won in our 110-game local run, by model turn; gray ticks mark where play ended. *Right*: every public scoring of identical code, including scorings after the deadline.

Two ideas moved tokens without adding any. Scheduler policies only redistribute a fixed supply: a simulator-derived policy scored 26.00 against 32.6 predicted. Linear probes on the residual stream detected wrong ARC-AGI-2 answers (AUROC 0.87 at layer 24), but extra turns for low-probe answers lost 1.85 points, as each re-read 9–41k evicted tokens. The judge's probability, read as an expectation over its digit tokens, already carried the signal (AUC 0.88).

8 Progress, generality, limitations and reproducibility

What would raise the scores. Eq. (1) separates what serving can do from what it cannot. On ARC-AGI-2, selection is near its ceiling: choosing perfectly among all answers registered in a 12-hour run would add only 1.5–3.3 points over ranking by the judge alone, and vrisk, T1n and harvesting already added 0.6–1.5. The distance to 85% is therefore mostly tasks that no conversation solved. Closing it needs a higher token worth: a stronger base model, or better perception, the failure we found when reading traces (§4). On ARC-AGI-3, 41 of the 42 games not won had cleared a level and then stalled (Figure 3), so more tokens alone also give diminishing returns there. We do not see a path to either goal from serving alone. What serving adds is a multiplier on w. The levers and the evaluation protocol carry over to a stronger open model, which would receive more tokens on the same hardware, as ours did.

Generality. Nothing in the serving stack is ARC-specific. Any long-horizon agent on a fixed GPU budget is bounded by Eq. (1), and the levers transfer to other MoE agents: pruning calibrated on the agent's own traces, 4-bit KV, batch-aware routing, draft-head retraining, restricted-vocabulary sampling with exactness checks, state reuse for hybrid models, and conversation parking. Neither agent holds game- or task-specific knowledge beyond prompts and helpers, and test–judge–retry fits any problem with checkable examples.

Change	speed	token worth / score
ARC-AGI-3 mixed NVFP4-FP8 checkpoint (4-bit activations) under vLLM	+38% tok/s, with the engine and streams (10 → 14) also changed	beyond 60k tokens per game, a token worth 0.85 of an SGLang token; the LB comparison coincided with a scheduler change
ARC-AGI-3 OEA k_0=4 instead of 5	+2.2% (older configuration)	ΔKL +0.017; 58-game score −0.8 (SE 3.5); LB 42.97 vs 46.20, one draw each
ARC-AGI-3 dropping preserved reasoning history	—	~3× tokens per action; 58-game score −49 (−38 with trimming adjusted to the shorter prompts)
ARC-AGI-2 224 experts + self-distillation	9–15% per token	1.2–1.4× tokens per task; −4.5 in a 12-h emulation (90% CI −8.3, −0.8)
ARC-AGI-2 fine-tuning on a stronger model's traces	—	37.2% → 28.3%: learned to stop thinking early

Table 3: Changes that raised R or kept it but did not raise the score, mostly because w fell. A single LB draw, or a 58-game difference smaller than its SE, is within noise.

Limitations. Gains are bounded by the base model's reasoning; we did not change what an agent does on a task it cannot understand. Several components show clear speed gains but no significant score evidence. The 150-stream ARC-AGI-2 setting never ran a full 12 hours on L4 before submission. The ARC-AGI-2 notebook has no fallback model, and T1n can fill a slot with an out-of-range color.

Reproducibility. Both submitted notebooks are public, at the exact submitted versions, and run offline on Kaggle hardware:

- ARC-AGI-2, submission 57852801 (the main notebook): `refv00/arc2-x-pr177-c150-cpu64-submission-12h-t1nfix`, version 4, on 4×L4.
- ARC-AGI-3, submission 57904278: `pompom555/arc3-pr46-v7-submit-20261102`, version 1, on one RTX PRO 6000.

Acknowledgements

We thank tk256ailab, ttyn4519 and yuto0712, the other members of our ARC-AGI-3 team, for their help with that entry. tk256ailab and yuto0712 ran several of the ARC-AGI-3 measurements in Tables 1 and 3. We also thank Saad Ahmed for generously providing access to a GH200 server for development experiments, enabling us to test hypotheses more quickly.

References

[1] François Chollet, Mike Knoop, Gregory Kamradt, Bryan Landers, and Henry Pinkard. ARC-AGI-2: A new challenge for frontier AI reasoning systems. *arXiv preprint arXiv:2505.11831*, 2025.

[2] ARC Prize Foundation. ARC-AGI-3 and its scoring methodology. `https://arcprize.org/arc-agi/3`, `https://docs.arcprize.org/methodology`, 2026.

[3] François Chollet. On the measure of intelligence. *arXiv preprint arXiv:1911.01547*, 2019.

[4] Alexia Jolicoeur-Martineau. Less is more: Recursive reasoning with tiny networks. *arXiv preprint arXiv:2510.04871*, 2025.

[5] J. Pourcel, C. Colas, and P.-Y. Oudeyer. Self-improving language models for evolutionary program synthesis: A case study on ARC-AGI. ARC Prize 2025 Paper Award, second place, 2025.

[6] François Chollet, Mike Knoop, Gregory Kamradt, and Bryan Landers. ARC Prize 2024: Technical report. *arXiv preprint arXiv:2412.04604*, 2024.

[7] Ekin Akyürek, Mehul Damani, Linlu Qiu, Han Guo, Yoon Kim, and Jacob Andreas. The surprising effectiveness of test-time training for abstract reasoning. *arXiv preprint arXiv:2411.07279*, 2024.

[8] Daniel Franzen, Jan Disselhoff, and David Hartmann. Product of experts with LLMs: Boosting performance on ARC is a matter of perspective. In *International Conference on Machine Learning (ICML)*, 2025.

[9] Ivan Sorokin and Jean-François Puget. NVARC solution to ARC-AGI-2 2025. ARC Prize 2025, first place on the Kaggle leaderboard, 2025.

[10] François Chollet, Mike Knoop, Gregory Kamradt, and Bryan Landers. ARC Prize 2025: Technical report. *arXiv preprint arXiv:2601.10904*, 2026.

[11] Tufa Labs. The duck: ARC-AGI-3 inference harness, winning solution to ARC-AGI-3 milestone 1. `https://github.com/Tufalabs/duck-harness`, 2026.

[12] Daniel Franzen. ARC-AGI-3 milestone 2 solution. Public Kaggle notebook, ARC Prize 2026 ARC-AGI-3 competition, 2026.

[13] Woosuk Kwon et al. Efficient memory management for large language model serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles (SOSP)*, 2023.

[14] Yaniv Leviathan, Matan Kalman, and Yossi Matias. Fast inference from transformers via speculative decoding. In *International Conference on Machine Learning (ICML)*, 2023.

[15] Fabian Gloeckle, Badr Youbi Idrissi, Baptiste Rozière, David Lopez-Paz, and Gabriel Synnaeve. Better & faster large language models via multi-token prediction. *arXiv preprint arXiv:2404.19737*, 2024.

[16] Zirui Liu, Jiayi Yuan, Hongye Jin, Shaochen Zhong, Zhaozhuo Xu, Vladimir Braverman, Beidi Chen, and Xia Hu. KIVI: A tuning-free asymmetric 2bit quantization for KV cache. *arXiv preprint arXiv:2402.02750*, 2024.

[17] Elias Frantar, Roberto L. Castro, Jiale Chen, Torsten Hoefler, and Dan Alistarh. MARLIN: Mixed-precision auto-regressive parallel inference on large language models. *arXiv preprint arXiv:2408.11743*, 2024.

[18] Ant Group and the vLLM community. Announcing Humming: Fast, flexible kernels for quantized inference. vLLM blog, `https://vllm.ai/blog/2026-10-21-humming`, 2026.

[19] Mike Lasby, Ivan Lazarevich, Nish Sinnadurai, Sean Lie, Yani Ioannou, and Vithursan Thangarasa. REAP the experts: Why pruning prevails for one-shot MoE compression. *arXiv preprint arXiv:2510.13999*, 2025.

[20] Costin-Andrei Oncescu et al. Opportunistic expert activation: Batch-aware expert routing for faster decode without retraining. *arXiv preprint arXiv:2511.02237*, 2025.

[21] Qwen Team. Qwen3.8-Flash-Next. Open-weight model release, Qwen Community License 1.0, 2026.

[22] Songlin Yang, Jan Kautz, and Ali Hatamizadeh. Gated delta networks: Improving Mamba2 with delta rule. In *International Conference on Learning Representations (ICLR)*, 2025.